

Final Capstone Project: Production-Ready Customer Support Agent

Project Summary

Build a production-style **Customer Support Agent** for a growing software business. The assistant should help customers answer product and billing questions, troubleshoot common issues, plan onboarding next steps, and escalate urgent requests when automation is not enough.

This capstone combines the full Agentic AI Bootcamp:

- Week 1: LLM agents, tool calling, environment variables, API integrations, and streaming.
- Week 3: typed state, LangGraph nodes, conditional routing, graph compilation, and workflow visualization.
- Week 4: structured output, Pydantic schemas, planning workflows, review loops, and revision.
- Week 5: embeddings, vector stores, retrievers, similarity search, and retrieval inspection.
- Week 6: full RAG workflow with routing, grounding, source-aware answering, and answer review.
- Week 7: short-term and long-term memory.
- Week 8: production system design with FastAPI backend, service modules, schemas, vector-store services, API routes, CORS, and a frontend chat experience.

The project should be medium difficulty. You are not expected to build enterprise authentication, payments, deployment automation, or a complex CRM. You are expected to design a clean, working system that looks and behaves like the first version of a real production AI support assistant.

Scenario

You have been hired by a SaaS company called **Northstar CRM**, a customer relationship management platform for small businesses. Northstar CRM customers ask questions such as:

- "How do I import contacts from a CSV file?"
- "What is included in the Pro plan?"
- "Why am I seeing duplicate contacts?"
- "I was charged twice this month. What should I do?"
- "My team cannot log in and we have a sales demo in one hour."
- "Help me create an onboarding checklist for my sales team."
- "How do I connect my email account?"

The assistant must:

1. Decide whether the user needs knowledge-base retrieval, troubleshooting, planning, billing/support escalation, or a general answer.
2. Retrieve grounded information from local product, billing, and support documents when needed.
3. Generate a helpful answer with source citations.
4. Review the answer for grounding, completeness, and support risk.

5. Use short-term memory to preserve the current conversation.
6. Use long-term memory to remember stable customer facts or preferences.
7. Return clear API responses through a FastAPI backend.
8. Provide a usable frontend chat interface.
9. Include production-minded documentation, test cases, and architecture notes.

Required Product

Build a web-based AI support assistant with two main components:

Backend

Create or extend a FastAPI backend inspired by the Week 8 backend structure.

Recommended structure:

```
capstone/
  backend/
    app/
      main.py
    api/
      chat_route.py
      health_route.py
    graph/
      customer_support_graph.py
    schemas/
      chat_schema.py
    services/
      document_loader.py
      vector_store.py
      memory_store.py
      support_tools.py
    tests/
      test_graph_routes.py
      test_api_contract.py
  knowledgebase/
    product_overview.txt
    pricing_and_billing.txt
    account_access_guide.txt
    integrations_guide.txt
    troubleshooting_playbook.txt
    escalation_policy.txt
```

Frontend

Create or extend a Next.js frontend inspired by Week 8.

Minimum frontend requirements:

- Textarea for the customer question.
- Submit button with loading state.
- Answer panel.
- Route/status panel showing what path the graph selected.
- Source cards when retrieval was used.
- Review/confidence panel showing whether the answer passed grounding review.
- Escalation or assigned-team panel for support cases.
- Clear error message when the backend is unavailable.

Required LangGraph Workflow

Your backend must include a LangGraph workflow with typed state and conditional routing.

At minimum, your graph must include these paths:

flowchart TD

```

Start([START]) --> Classify[classify_request]
Classify --> Router{route_request}
Router --> |knowledge| Retrieve[retrieve_documents]
Router --> |troubleshooting| Troubleshoot[generate_troubleshooting_steps]
Router --> |escalation| Escalate[prepare_escalation]
Router --> |onboarding| Onboarding[generate_onboarding_plan]
Router --> |general| GeneralAnswer[general_answer]
Retrieve --> KnowledgeAnswer[generate_grounded_answer]
KnowledgeAnswer --> Review[review_answer]
Troubleshoot --> Review
Escalate --> Review
Onboarding --> Review
GeneralAnswer --> Review
Review --> ReviewRouter{review_decision}
ReviewRouter --> |revise| Revise[revise_answer]
ReviewRouter --> |finalize| Finalize[finalize_response]
Revise --> Review
Finalize --> SaveMemory[save_memory]
SaveMemory --> End([END])

```

You may add nodes, but do not remove the core behavior.

Required State Schema

Use TypedDict or a Pydantic state model. Your state must include at least:

Field	Type	Purpose
question	str	Customer's latest message
customer_id	str	Stable customer or account identifier

Field	Type	Purpose
thread_id	str	Conversation/session identifier
messages	list	Short-term conversation messages
route	Literal[...]	Selected path: knowledge, troubleshooting, escalation, onboarding, or general
documents	list[Document]	Retrieved documents
answer	str	Draft answer
review	structured object or str	Grounding and quality review
confidence	float or int	Confidence score
sources	list	Source metadata returned to the frontend
priority	Literal["low", "normal", "urgent"]	Support priority for operational issues
assigned_team	str	Team responsible for follow-up
memory_updates	dict or list	Long-term customer facts to save
final_answer	str	Final response returned by API

Required Features

1. Intent Classification and Routing

Classify each customer message into one of five routes:

- **knowledge:** product features, pricing, plan limits, integrations, settings, documentation questions.
- **troubleshooting:** login problems, duplicate data, imports failing, email sync not working, workflow automation issues.
- **escalation:** billing disputes, urgent outages, locked accounts, security concerns, angry customer requests, time-sensitive business impact.
- **onboarding:** setup plans, team rollout, migration checklist, training plan, adoption planning.
- **general:** safe general questions unrelated to Northstar CRM.

Use a typed router with `Literal[...]` return values and `add_conditional_edges`.

2. Retrieval-Augmented Generation

Implement local knowledge-base retrieval using:

- Document
- text splitting
- embeddings
- Chroma or another local vector store
- retriever with k between 3 and 6

Grounded answers must:

- Use retrieved context only for product, billing, and policy-specific answers.
- Mention source documents.
- Avoid inventing product capabilities, plan limits, billing rules, or escalation promises not found in the documents.
- Return a fallback if no relevant context exists.

3. Review and Revision Loop

Add a review node that checks:

- Did the answer actually address the customer question?
- For knowledge answers, is the answer supported by retrieved context?
- Are source references present when needed?
- Does the response avoid unsupported refund, legal, security, or SLA promises?
- Is the answer clear and actionable?
- Should the answer be revised?

Allow at least one revision cycle, then finalize.

4. Escalation Path

Build an escalation branch for urgent or sensitive support issues.

Examples:

- "I was charged twice this month."
- "My team cannot log in and we have a sales demo in one hour."
- "I think someone accessed our account without permission."
- "Our email sync has been down all day and we are losing deals."

The branch must produce:

- priority level: low, normal, or urgent
- assigned team, such as `billing_support`, `technical_support`, `security_team`, `customer_success`, or `engineering_on_call`
- recommended next steps
- escalation language when needed
- information the human team should collect next

This path should reuse the routing and triage ideas from the Week 3 assignment.

5. Troubleshooting Path

Build a troubleshooting branch for product issues that can usually be handled without immediate escalation.

The branch must:

- Identify the likely issue category.
- Ask for missing details when necessary.
- Provide ordered troubleshooting steps.
- Include "when to escalate" criteria.
- Use retrieved docs when the issue depends on documented product behavior.

6. Onboarding and Planning Path

Build an onboarding branch inspired by Week 4.

The onboarding branch must:

- Extract structured details from the user request.
- Identify missing information.
- Generate a practical onboarding or rollout plan.
- Include assumptions, checklist, risks, owners, and next actions.
- Use the review node to improve the plan when necessary.

Optional: use a tool for deadline calculations, account-size estimates, or implementation timeline planning.

7. Short-Term Memory

Use LangGraph checkpointing or another session-level memory approach so a customer can ask follow-up questions in the same session.

Example:

Customer: "How do I import contacts from CSV?"

Assistant: answers with source.

Customer: "What if some rows fail?"

Assistant: understands that "rows" refers to the CSV import process.

8. Long-Term Memory

Store at least three stable customer facts when they appear naturally:

- company type or industry
- plan tier, such as Starter, Pro, or Enterprise
- team size
- preferred support tone or response style
- active goal, such as migration, onboarding, cleanup, or renewal

The assistant should use these facts later when helpful, but should not over-personalize every answer.

9. FastAPI API Contract

Expose at least:

- GET /health
- POST /chat

POST /chat request should include:

```
{
  "customer_id": "acct-123",
  "thread_id": "session-abc",
  "question": "I was charged twice this month. What should I do?"
}
```

POST /chat response should include:

```
{
  "question": "I was charged twice this month. What should I do?",
  "route": "escalation",
  "answer": "...",
  "review": "...",
  "confidence": 0.87,
  "priority": "urgent",
  "assigned_team": "billing_support",
  "sources": [
    {
      "source": "pricing_and_billing.txt",
      "content_preview": "Billing disputes should be reviewed..."
    }
  ],
  "memory_updates": []
}
```

Use Pydantic request and response schemas.

10. Frontend Chat Experience

The frontend must call the backend and display:

- final answer
- selected route
- confidence or review status
- sources
- priority and assigned team when present
- loading state
- error state

Design does not need to be complex, but it should feel intentional and usable.

11. Observability and Debugging

Add one or more of:

- graph streaming logs
- node-level audit trail
- route and review fields in the API response
- backend logging for route, retrieved source count, priority, assigned team, and review status
- README troubleshooting section

Required Knowledge Base

Create a local Northstar CRM knowledge base. You may write the documents yourself.

Include at least six .txt files, such as:

- product_overview.txt
- pricing_and_billing.txt
- account_access_guide.txt
- integrations_guide.txt
- troubleshooting_playbook.txt
- escalation_policy.txt

Optional additions:

- csv_import_guide.txt
- email_sync_guide.txt
- security_policy.txt
- onboarding_checklist.txt
- sla_and_support_hours.txt

Your retrieval tests should prove that the documents are actually being used.

Required Test Scenarios

Create a small test script or test file that runs at least ten scenarios:

Test	Example Question	Expected Route
1	How do I import contacts from a CSV file?	knowledge or troubleshooting
2	What is included in the Pro plan?	knowledge
3	How do I connect my email account?	knowledge

Test	Example Question	Expected Route
4	Why am I seeing duplicate contacts after import?	troubleshooting
5	My team cannot log in and we have a sales demo in one hour.	escalation
6	I was charged twice this month.	escalation
7	I think someone accessed our account without permission.	escalation
8	Help me create a 30-day onboarding plan for a five-person sales team.	onboarding
9	We are migrating from spreadsheets. What setup checklist should we follow?	onboarding
10	Tell me a harmless joke about sales pipelines.	general

For each test, print or assert:

- route
- final answer exists
- review status exists
- source count for knowledge route
- priority and assigned team for escalation route
- ordered steps for troubleshooting route
- no sources for general route

Required Deliverables

Submit:

1. Working backend source code.
2. Working frontend source code.
3. Customer-support knowledge-base documents.
4. Graph visualization image or Mermaid diagram.
5. README with setup and run instructions.
6. Architecture section explaining the graph, API, retrieval, memory, and frontend.
7. Test scenarios and results.
8. A short demo video or screenshots showing:
 - a product/billing answer with sources
 - a troubleshooting response
 - an escalation response with priority and assigned team
 - an onboarding plan
 - a follow-up question using memory

Suggested Implementation Plan

Phase 1: Backend Skeleton

- Create FastAPI app.
- Add `/health`.
- Add `/chat` with Pydantic schemas.
- Return a placeholder response.

Phase 2: Knowledge Base and Retriever

- Write Northstar CRM `.txt` documents.
- Load documents.
- Split documents.
- Create vector store.
- Build retriever.
- Test retrieval manually.

Phase 3: LangGraph Workflow

- Define typed state.
- Add classifier node.
- Add conditional routing.
- Add knowledge branch.
- Add general branch.
- Compile and invoke the graph.
- Save graph visualization.

Phase 4: Troubleshooting, Escalation, and Onboarding Branches

- Add troubleshooting branch.
- Add escalation branch.
- Add priority/team assignment.
- Add structured onboarding extraction.
- Add review and revision loop.

Phase 5: Memory

- Add short-term memory by `thread_id`.
- Add long-term memory by `customer_id`.
- Add at least one follow-up test.

Phase 6: Frontend

- Build chat interface.
- Display answer, route, confidence/review, sources, priority, and assigned team.
- Handle loading and errors.

Phase 7: Testing and Documentation

- Run the required ten scenarios.
- Document setup.
- Add architecture notes.
- Add screenshots or demo video.

Evaluation Rubric

Criterion	Points	Evidence
LangGraph workflow	20	Typed state, conditional routing, multiple branches, review/revision, graph visualization
RAG implementation	15	Document loading, chunking, embeddings, vector store, retriever, source-grounded answers
Production API design	15	FastAPI structure, Pydantic schemas, /health, /chat, clear response contract
Frontend experience	10	Usable chat UI, loading/error states, answer display, source cards, route/review/team display
Memory	10	Short-term thread memory and long-term customer memory used correctly
Escalation, troubleshooting, and onboarding branches	10	Practical escalation, support troubleshooting, and structured planning paths
Testing	10	Required scenarios, route checks, source checks, escalation checks, memory follow-up
Documentation and architecture	10	Setup instructions, architecture explanation, tradeoffs, demo evidence
Total	100	

Stretch Goals

Optional extensions:

- Add a confidence threshold that forces fallback or escalation.
- Add a human handoff queue stored in a local JSON or SQLite file.
- Add a feedback button in the frontend.
- Add streaming responses.
- Add file upload for new support documents.
- Add a small admin page showing recent questions, routes, reviews, and assigned teams.
- Add Docker files for backend and frontend.
- Add LangSmith or another tracing approach if you have access.

- Add a mock ticket-creation tool that creates a local ticket record.

Constraints

- Do not hard-code answers for the required test questions.
- Do not answer product, billing, or policy questions without retrieval.
- Do not hide review failures; expose them clearly in the response.
- Do not promise refunds, credits, SLA remedies, security conclusions, or engineering fixes unless the knowledge base supports the claim.
- Do not store secrets in source code.
- Do not require paid infrastructure beyond the model/embedding API you already use for class.
- Keep the first version focused. A clean, working production-style assistant is better than an unfinished large system.

Final Reflection

Include a short reflection answering:

1. Which part of the graph was hardest to design?
2. How did you decide when to use retrieval versus troubleshooting versus escalation versus onboarding?
3. What did your review node catch or improve?
4. How does memory improve the customer support experience?
5. What would you change before deploying this to real customers?